

Report Approval

The project work "**UniCare (AI Powered Issue Resolution Platform)**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Prof. Ankit Gupta
Assistant Professor
Medicaps University

Internal Examiner

Prof. Laxmi Kag
Assistant Professor
Medicaps University

Declaration

I hereby declare that the project entitled "UniCare (AI Powered Issue Resolution Platform)" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, completed under the supervision of **Prof. Ankit Gupta & Prof. Laxmi Kag**, Faculty of Engineering, Mediacaps University Indore, is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Vansh Patil

B.Tech. III Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Certificate

We, **Prof. Ankit Gupta & Prof. Laxmi Kag** , certify that the project entitled "**UniCare (AI Powered Issue Resolution Platform)**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Vansh Patil [EN23CS3011110]** is the record carried out under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Ankit Gupta

Department of Computer Science and Engineering
Medicaps University, Indore

Prof. Laxmi Kag

Department of Computer Science and Engineering
Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department
Computer Science & Engineering
Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medcaps University, whose unfailing support and enthusiasm has always boosted up my morale.

I also thank Prof. (Dr.) Ratnesh Litoriya, Associate Dean, Faculty of Computing Engineering, Medcaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department Dr. Kailash Chandra Bandhu for his continuous encouragement for the betterment of the project.

I am deeply thankful to my project guide, [Your Faculty Guide Name], for their invaluable guidance, continuous support, and expert advice throughout the development of UniCare. Their technical insights and encouragement at every stage of this project were instrumental in its successful completion.

It is their help and support, due to which I became able to complete the design and technical report. Without their support this report would not have been possible.

Vansh Patil

B.Tech. III Year

Department of Computer Science & Engineering

Faculty of Engineering

Medcaps University, Indore

Abstract

In contemporary academic institutions, the management of student grievances and administrative complaints remains a largely manual, paper-based process characterised by inefficiencies in categorisation, routing, and resolution tracking. Complaints are submitted through physical forms or generic email systems, routed manually to relevant departments, and tracked through spreadsheets or informal communication channels. This fragmented workflow results in delayed acknowledgements, inconsistent prioritisation, and poor resolution transparency for students.

UniCare is an AI-powered complaint and issue resolution platform specifically designed for university environments to address these systemic inefficiencies. The platform enables students to digitally submit grievances through a structured web interface, where complaints are automatically classified by category and prioritised by urgency using an integrated Artificial Intelligence microservice. The AI pipeline employs Natural Language Processing techniques, including TF-IDF vectorisation and Logistic Regression classification, to analyse complaint text and assign it to one of five predefined categories: Academic, Infrastructure, Hostel, Administration, or Other, along with a priority level of Low, Medium, or High .

The system architecture follows a microservices design pattern comprising four tiers: a React.js single-page application frontend, a Node.js and Express.js REST API application server, a Python FastAPI AI microservice for NLP-based inference, and a MongoDB Atlas cloud database for persistent storage. JWT-based stateless authentication enforces role-based access control, providing distinct interface experiences for Students, Faculty, and Administrators. The administrator panel presents a priority-sorted complaint queue with resolution, reassignment, and escalation capabilities, with all actions triggering real-time notifications to submitting students.

The platform was developed using the Agile Incremental model across four structured sprints, covering authentication, complaint submission with AI integration, admin resolution workflows, and analytics respectively. UniCare eliminates manual classification overhead, reduces average complaint response time, and provides comprehensive audit trails for institutional governance. The system is cloud-ready and designed for deployment in any university environment seeking to modernise its grievance redressal infrastructure.

Keywords: AI, NLP, Complaint Management, TF-IDF, Logistic Regression, FastAPI, React.js, Node.js, MongoDB, JWT, Role-Based Access Control, Microservices, University Grievance Syste

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgements	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	viii
	List of Tables	x
	Abbreviations	xi
Chapter 1	Introduction	
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	3
	1.4 Significance	3
	1.5 Research Design	4
	1.6 Chapter Scheme	4
Chapter 2	Requirements Specification	
	2.1 User Characteristics	5
	2.2 Functional Requirements	6
	2.3 Dependencies	8
	2.4 Performance Requirements	9
	2.5 Hardware Requirements	9

		Page No.
	2.6 Constraints & Assumptions	10
Chapter 3	Design	
	3.1 System Architecture	11
	3.2 Use Case Diagram	12
	3.3 Activity Diagram	13
	3.4 Sequence Diagram	14
	3.5 Class Diagram	15
	3.6 Database Design	16
	3.7 Component Design	18
Chapter 4	Implementation, Testing and Maintenance	
	4.1 Languages, IDEs and Technologies	20
	4.2 Testing Techniques and Test Plans	22
	4.3 Installation Instructions	24
	4.4 End User Instructions	24
Chapter 5	Results and Discussions	25
	5.1 User Interface Representation	26
	5.2 Module Descriptions	27
	5.3 Database Tables	28
Chapter 6	Summary and Conclusions	29
Chapter 7	Future Scope	30
	Appendix — Glossary	31
	Bibliography	32

List of Figures

Figure No.	Title	Location
1	Figure 3.1: UniCare – System Architecture Diagram	Chapter 3
2	Figure 3.2: UniCare – Use Case Diagram	Chapter 3
3	Figure 3.3: UniCare – Activity Diagram: Complaint Lifecycle	Chapter 3
4	Figure 3.4: UniCare – Sequence Diagram: Complaint Submission & Resolution	Chapter 3
5	Figure 3.5: UniCare – Class Diagram	Chapter 3
6	Figure 3.6: UniCare – Package Diagram	Chapter 3
7	Figure 3.7: UniCare – Component Diagram	Chapter 3
8	Figure 3.8: UniCare – Deployment Diagram	Chapter 3
9	Figure 3.9: UniCare – ER Diagram	Chapter 3
10	Figure 4.1: AI Classification Pipeline – TF-IDF + Logistic Regression	Chapter 4
11	Figure 5.1: Login Page – UniCare	Chapter 5
12	Figure 5.2: Student Dashboard	Chapter 5
13	Figure 5.3: Raise Complaint Page	Chapter 5
14	Figure 5.4: Admin Complaint Queue Panel	Chapter 5

List of Tables

Table No.	Title	Location
1	Table 2.1: Functional Requirements – UniCare	Chapter 2
2	Table 2.2: Non-Functional Requirements	Chapter 2
3	Table 2.3: Hardware and Software Requirements	Chapter 2
4	Table 3.1: UniCare Data Dictionary	Chapter 3
5	Table 3.2: Interface Screen Descriptions	Chapter 3
6	Table 4.1: Test Cases – Unit Testing	Chapter 4
7	Table 4.2: Test Cases – System Testing	Chapter 4
8	Table 5.1: Database Collections – MongoDB	Chapter 5

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
TF-IDF	Term Frequency – Inverse Document Frequency
ML	Machine Learning
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
SPA	Single Page Application
ODM	Object Document Mapping
UML	Unified Modelling Language
ER	Entity Relationship
HTTP/HTTPS	Hypertext Transfer Protocol / Secure
SDS	Software Design Specification
UI	User Interface
RBAC	Role-Based Access Control
CSE	Computer Science and Engineering

Notations & Symbols

Symbol / Notation	Meaning
<<include>>	UML relationship: mandatory included use case
<<extend>>	UML relationship: optional extended use case
1..*	One to many multiplicity in class diagram
PK	Primary Key – unique identifier in a collection
FK	Foreign Key – reference to another collection
POST / GET / PUT / DELETE	HTTP Methods used in REST API design
{ }	JSON object notation
[]	Array notation
ObjectId	MongoDB 12-byte unique document identifier

Chapter 1 – Introduction

1.1 Introduction

In the modern academic environment, the efficient management of student and staff grievances is an essential administrative responsibility. Universities across India handle hundreds of complaints weekly spanning academic disputes, infrastructure deficiencies, hostel management issues, and administrative irregularities. The traditional complaint management process, which relies predominantly on manual submission, paper-based routing, and informal email communication, is fundamentally ill-equipped to manage this volume with the transparency, speed, and accountability that students and faculty rightfully expect.

UniCare is an AI-powered issue resolution platform developed specifically to address this problem in university settings. By integrating Artificial Intelligence, Natural Language Processing, and a microservices architecture, UniCare automates the classification, prioritisation, and routing of complaints, enabling administrative teams to respond more efficiently while providing students with real-time visibility into the status of their grievances.

1.2 Problem Statement

The existing complaint management infrastructure in most academic institutions suffers from several critical deficiencies. First, there is no standardised digital channel for complaint submission; students resort to physical forms, personal emails, or verbal escalation, resulting in complaints going untracked or unacknowledged. Second, manual classification of complaints by administrative staff is inconsistent, time-consuming, and prone to routing errors that delay resolution. Third, the absence of priority assessment mechanisms means that urgent complaints, such as safety-related infrastructure failures, are treated with the same urgency as low-severity academic queries. Fourth, students have no mechanism to track the status of their submitted complaints, leading to repeated follow-ups and increased administrative workload.

These inefficiencies collectively result in poor student satisfaction, unresolved grievances, and a lack of accountability in institutional governance. UniCare directly addresses each of these pain points through automation, structured workflows, and AI-assisted decision-making.

1.3 Literature Review

Several existing systems have attempted to address complaint management in institutional and service environments. Traditional Helpdesk systems such as Zendesk and Freshdesk provide ticketing functionality for corporate environments but lack contextual understanding of academic complaint types and do not offer AI-based classification tailored to university grievance categories. Government grievance portals such as CPGRAMS in India provide a web-based submission interface but are designed for government departments and are not adaptable to the specific hierarchical and categorical structure of university complaints.

Research in the domain of automated text classification has demonstrated that TF-IDF vectorisation combined with Logistic Regression achieves competitive accuracy on short-text classification tasks with limited training data, making it particularly suitable for institutional complaint datasets. Studies by Sebastiani (2002) on text categorisation and subsequent work on multi-class NLP classification confirm that this approach provides a practical baseline for complaint routing systems where interpretability and deployment simplicity are priorities.

Microservices architecture, as described by Newman (2015), enables the AI classification component to be developed, deployed, and scaled independently of the main application backend, an architectural pattern particularly valuable when the AI model requires periodic retraining without disrupting the core complaint management service. UniCare incorporates these research-validated techniques into a cohesive, deployable platform designed for the specific constraints of Indian university environments.

1.4 Objectives

The primary objectives of the UniCare project are as follows:

- To design and develop a web-based complaint submission platform that provides a structured, consistent digital interface for students, faculty, and administrators.
- To implement an AI-based complaint classification pipeline using TF-IDF vectorisation and Logistic Regression to automatically categorise and prioritise complaints without manual intervention.
- To architect a microservices-based system separating the AI inference service from the application backend, enabling independent scalability and maintainability.
- To develop role-based dashboards providing contextually appropriate interfaces for Students, Faculty, and Administrators with distinct permissions and views.
- To implement real-time complaint tracking and notification systems ensuring students are informed of status changes throughout the resolution lifecycle.
- To replace manual, paper-based grievance processes with an automated, traceable, and data-driven workflow that improves institutional accountability.

1.5 Significance

UniCare represents a significant advancement in institutional grievance management by integrating AI capabilities into a domain that has historically relied entirely on manual human judgment. The significance of this platform extends across multiple dimensions: operational efficiency is improved through automated complaint routing that eliminates manual classification steps; student experience is enhanced through real-time status tracking and transparent communication; institutional governance is strengthened through comprehensive audit trails and analytics on complaint patterns; and resource allocation is optimised by enabling administrators to prioritise high-urgency complaints effectively.

Furthermore, the open microservices architecture of UniCare means that the AI classification component can be iteratively improved as more institutional data becomes available, allowing the system to grow in accuracy and capability over time without requiring a complete architectural redesign.

1.6 Research Design

UniCare was developed using the Agile Incremental Software Process Model, which organises development into four iterative sprints. Sprint 1 delivered user authentication and role-based access control. Sprint 2 delivered the complaint submission form, AI

classification pipeline, and MongoDB integration. Sprint 3 delivered the admin resolution workflow, reassignment, escalation, and notification dispatch. Sprint 4 delivered the analytics dashboard, reporting module, and final UI polish. Each sprint culminated in a working software increment reviewed against defined acceptance criteria, allowing continuous refinement based on functional testing and stakeholder feedback.

The Object-Oriented design methodology was applied throughout, with the system modelled using UML artefacts including use case diagrams, activity diagrams, sequence diagrams, and class diagrams prior to implementation. The AI component was developed and validated using a representative dataset of university complaint texts, with classification accuracy measured using precision, recall, and F1 score metrics.

1.7 Chapter Scheme

Chapter 1 presents the introduction, problem statement, literature review, objectives, significance, and research design of the UniCare project. Chapter 2 covers the complete requirement specification including user characteristics, functional and non-functional requirements, hardware specifications, and system constraints. Chapter 3 presents the system design through UML diagrams, architectural design, database design, component design, and interface specifications. Chapter 4 describes the implementation details covering the technology stack, AI model working, API integration, and testing procedures. Chapter 5 presents the results and discussion including UI screenshots, module descriptions, and database tables. Chapter 6 presents the conclusions drawn from the project, followed by Chapter 7 which outlines the future scope for system enhancement.

Chapter 2 – Requirement Specification

2.1 User Characteristics

The UniCare platform serves three distinct user types, each with unique system interaction patterns and access privileges:

2.1.1 Student Users

Students are the primary users of the UniCare platform and the principal complainants in the system. A typical student user is a B.Tech, M.Tech, or MBA student enrolled at the institution with basic proficiency in web-based applications. Students interact with UniCare through the student dashboard to submit new complaints, view the status of existing complaints, and receive resolution notifications. Students do not have access to administrative functions, and their view of complaint data is restricted to their own submissions. Students are expected to use the platform from desktop or laptop browsers, with occasional mobile access.

2.1.2 Administrator Users

Administrators are the secondary users responsible for reviewing, resolving, reassigning, and escalating complaints through the UniCare admin panel. Administrators are institutional staff members with domain expertise corresponding to their assigned complaint categories, for example, the Hostel Warden handles hostel-related complaints while the Academic Registrar handles academic disputes. Administrators have elevated system privileges enabling them to modify complaint status, add resolution notes, override AI-assigned categories, and generate institutional reports.

2.1.3 Faculty Users

Faculty members interact with UniCare as both complaint submitters in cases of resource or infrastructure concerns, and as domain-level reviewers for academically categorised complaints. Faculty access is a role with permissions between standard student access and full administrative access, limited to viewing complaints relevant to their department.

2.2 Functional Requirements

The functional requirements of UniCare are categorised by system module:

ID	Module	Requirement
FR01	Authentication	The system shall allow users to register and log in using institutional email credentials secured with bcrypt password hashing.
FR02	Authentication	The system shall issue JWT tokens upon successful login and validate tokens on all protected API endpoints.
FR03	Authentication	The system shall enforce role-based access control distinguishing Student, Faculty, and Admin permissions.
FR04	Complaint Submission	The system shall provide a structured complaint submission form with title, description, and optional category fields.
FR05	AI Classification	The system shall automatically classify complaint text into predefined categories using the FastAPI AI microservice.
FR06	AI Classification	The system shall assign a priority level of Low, Medium, High, or Critical to each complaint based on NLP analysis.
FR07	Complaint Tracking	The system shall maintain a real-time status trail for each complaint with timestamps for Open, In Progress, Resolved, and Closed states.
FR08	Admin Panel	The system shall present administrators with a priority-sorted complaint queue filterable by status, category, and date.
FR09	Admin Actions	The system shall allow administrators to Resolve, Reassign, or Escalate any complaint from the admin panel.
FR10	Notifications	The system shall dispatch email and in-app notifications to relevant users on complaint creation, status change, and resolution.

Table 2.1: Functional Requirements – UniCare

2.3 Non-Functional Requirements

Attribute	Requirement
Performance	The system shall respond to standard API requests within 500ms under normal load. AI classification shall complete within 2 seconds.
Scalability	The microservices architecture shall support independent horizontal scaling of the AI service without affecting the application backend.

Security	All API endpoints shall be protected with JWT authentication. Passwords shall be hashed using bcrypt with a minimum cost factor of 10.
Availability	The system shall target 99.5% uptime in production deployment on cloud infrastructure.
Usability	The interface shall follow responsive design principles compatible with desktop and tablet viewports.
Maintainability	The codebase shall follow modular design principles enabling component replacement without full system redeployment.

Table 2.2: Non-Functional Requirements

2.4 Dependencies

UniCare has the following external and internal dependencies:

- React.js v18+ and Node.js v18+ as the primary frontend and backend runtimes.
- Python 3.10+ with FastAPI and Uvicorn for the AI microservice.
- MongoDB Atlas cloud service for persistent data storage.
- An SMTP-compatible email service provider for notification dispatch.
- JSON Web Token library for stateless authentication implementation.
- scikit-learn library for TF-IDF vectorisation and Logistic Regression model training.

2.5 Performance Requirements

The UniCare system is designed to handle a concurrent user base of up to 500 active sessions without performance degradation. Standard CRUD API operations must respond within 500 milliseconds on a hosted cloud instance. The AI classification microservice must return category and priority predictions within 2000 milliseconds of receiving a complaint description payload. The frontend must achieve an initial page load time of under 3 seconds on standard broadband connections. MongoDB queries on the Complaints collection must complete within 200 milliseconds using compound indexes on status, category, priority, and user_id fields.

2.6 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 1.5 GHz	Quad-core 2.5 GHz
RAM	2 GB	4 GB
Storage	10 GB HDD	20 GB SSD
Browser	Chrome/Firefox (latest)	Chrome v110+
Network	5 Mbps broadband	10 Mbps broadband
Server (Cloud)	1 vCPU / 512 MB RAM	2 vCPU / 1 GB RAM

Table 2.3: Hardware and Software Requirements

2.7 Constraints

- The AI classification microservice in Version 1.0 uses rule-based NLP. Full ML model deployment is planned for Version 2.0 pending training dataset curation.
- The system is designed for web deployment; a native mobile application is outside the current project scope.
- UniCare assumes complaints are submitted in English language. Multilingual NLP support is a future enhancement.
- Institutional email domain validation for registration is configurable but not enforced by default in the development build.

Chapter 3 – Design

3.1 System Architecture

UniCare employs a four-tier layered microservices architecture designed for modularity, independent scalability, and maintainability. The four tiers are: Presentation Tier, Application Tier, Intelligence Tier, and Data Tier. All inter-tier communication occurs exclusively over HTTPS using RESTful APIs, ensuring a loosely coupled system design where each tier can be scaled and deployed independently without disrupting the others.

3.1.1 System Architecture Diagram

The system architecture diagram below (Figure 3.1 from the UniCare SDS) illustrates the four-tier design: the Presentation Layer (React.js SPA with Tailwind CSS and Shadcn/UI), the Application Layer (Node.js + Express REST API with JWT Middleware), the AI/Intelligence Layer (Python FastAPI NLP Microservice), and the Data Layer (MongoDB Atlas).

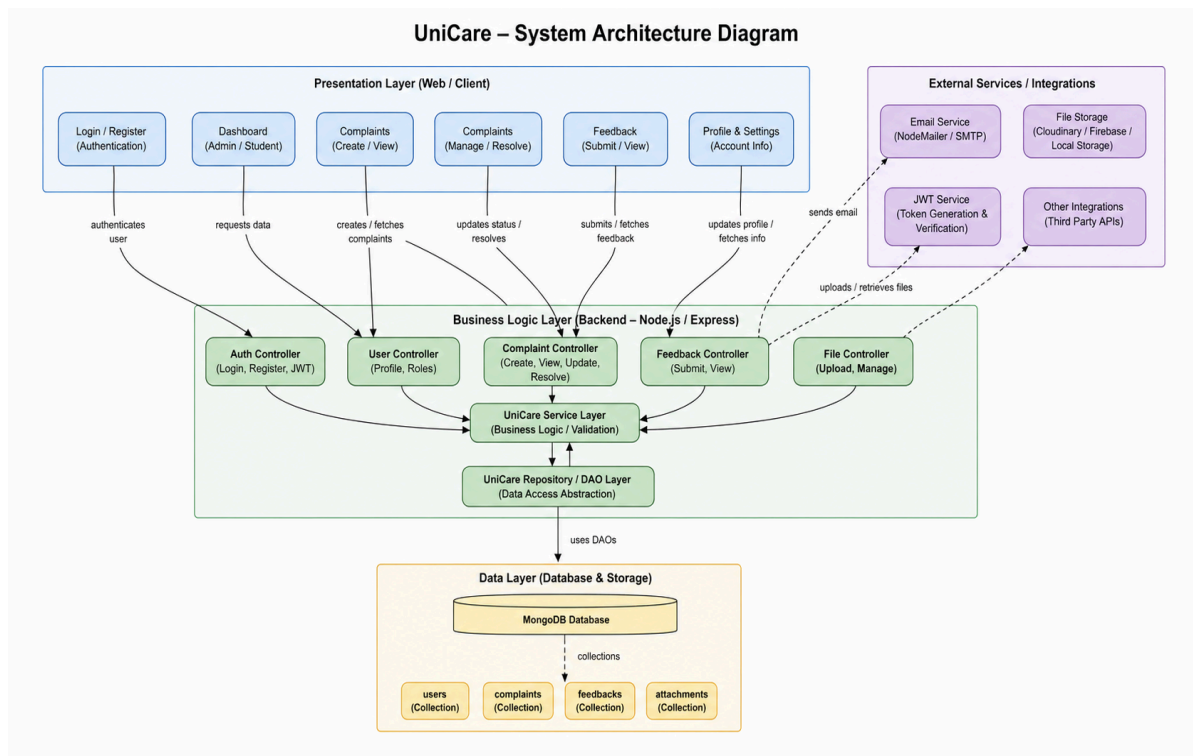


Figure 3.1: UniCare – System Architecture Diagram

3.1.2 Description of Architectural Design

Client Layer: The presentation tier is implemented as a React.js single-page application built with Vite and styled using Tailwind CSS with Shadcn/UI component library. The client communicates with the backend exclusively through RESTful HTTPS API calls. React Router v6 manages client-side navigation, and React Query handles data fetching, caching, and synchronisation. Role-based UI rendering ensures that Students and Admins receive contextually appropriate, distinct interface views.

Application Layer: The application tier is a Node.js and Express.js REST API server responsible for user authentication using JWT-based stateless tokens, complaint CRUD operations, business logic enforcement, and notification dispatch. JWT middleware validates all protected routes and extracts the user role for authorisation decisions.

AI / Intelligence Layer: A Python FastAPI microservice handles all AI inference tasks as an independently deployable service. The Version 1.0 implementation uses rule-based NLP with keyword matching and regular expressions to classify complaints into predefined categories.

Data Layer: MongoDB Atlas serves as the primary data store with separate collections for Users, Complaints, and Notifications. MongoDB's flexible document model accommodates the variable structure of complaint data and supports efficient querying using compound indexes on status, category, priority, and user ID fields. Mongoose ODM is used by the Node.js application layer for schema definition, data validation, and query building.

3.2 Use Case Diagram

The UniCare system supports three primary actors: Student (the primary actor who submits and tracks complaints), Admin (the secondary actor who manages and resolves complaints), . Key use cases include Raise Complaint, Track Complaint Status, View Notifications, Manage Complaint, Assign/Escalate, Generate Reports, and AI Auto-Classification.

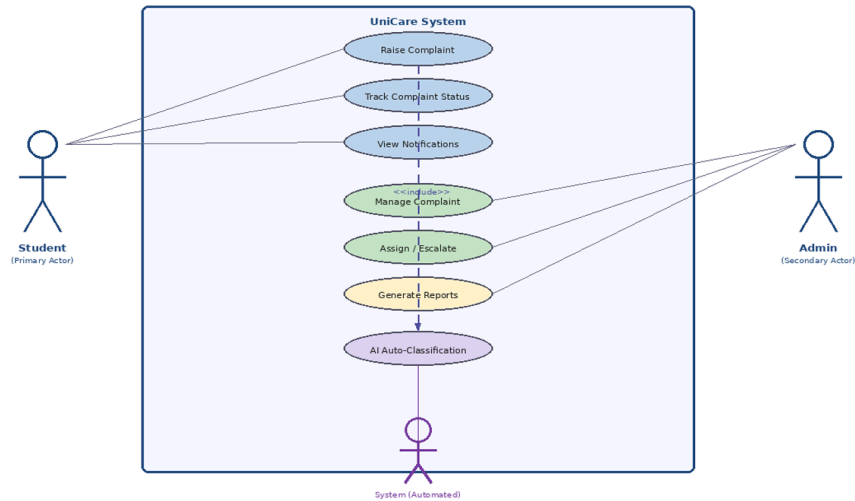


Figure 3.2: UniCare – Use Case Diagram

3.3 Activity Diagram

The Activity Diagram illustrates the end-to-end complaint lifecycle in UniCare: the Student logs in and navigates to the submission form; the system validates input looping back on error; the validated text is forwarded to the FastAPI AI classification service; category and priority are auto-assigned; the complaint is stored in MongoDB; the Admin is notified and takes a management action; and finally the Student receives a resolution notification and the complaint is marked resolved.

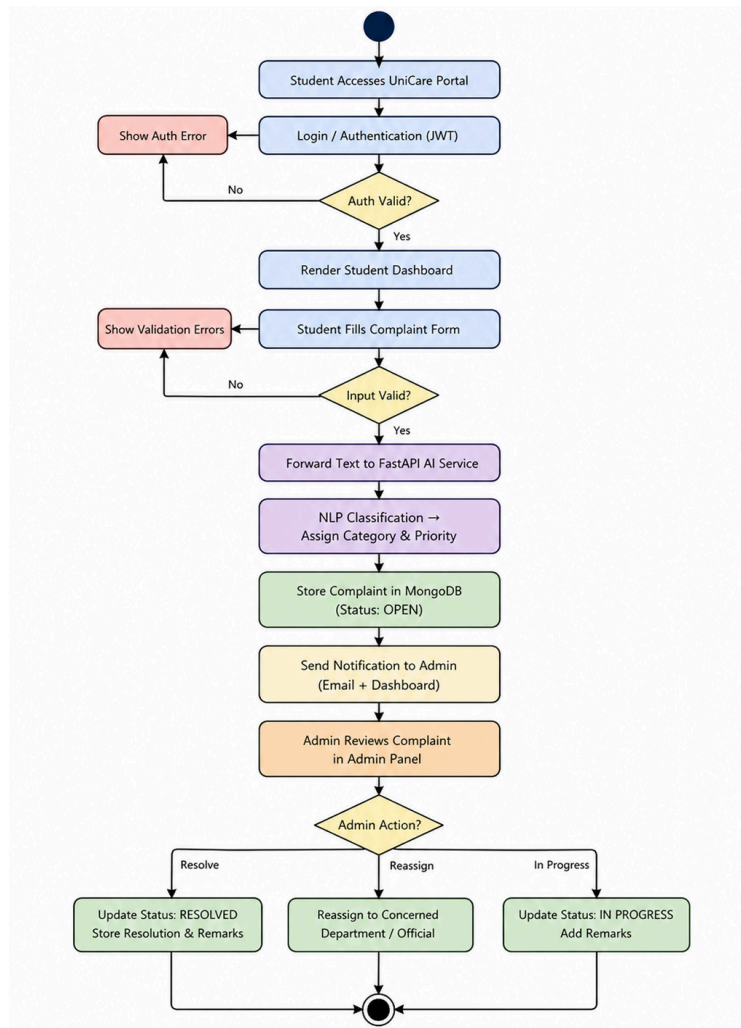


Figure 3.3: UniCare – Activity Diagram: Complaint Lifecycle

3.4 Sequence Diagram

The Sequence Diagram illustrates the Complaint Submission and Resolution flow across six objects: User (Browser), React Frontend, Node.js Backend, FastAPI AI Service, MongoDB Database, and Admin Panel. The student submits a complaint form; the frontend dispatches a POST request to the backend; the backend forwards the text to the AI service for classification; the classified result is stored in MongoDB; the admin is notified; the admin updates the complaint status; and the student is notified of the resolution outcome.

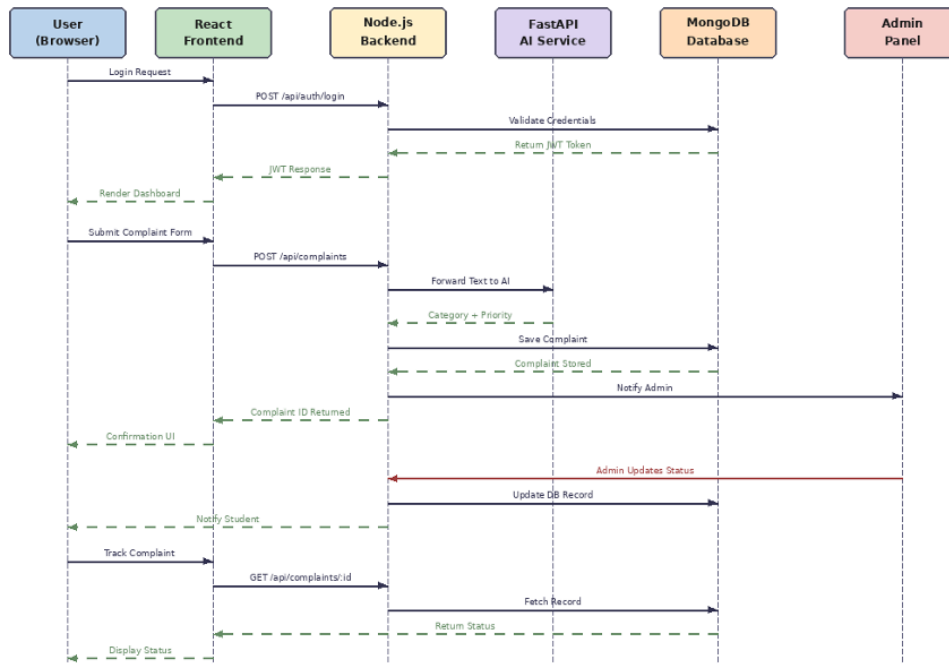


Figure 3.4: UniCare – Sequence Diagram: Complaint Submission & Resolution

3.5 Class Diagram

The Class Diagram shows five core classes: User, Complaint, Admin, Notification, and AIService, with their attributes, methods, and relationships. A User (1) submits many Complaints (1..*); each Complaint is managed by one Admin; a User receives many Notifications; and the AIService is associated with Complaints through the automated classification process.

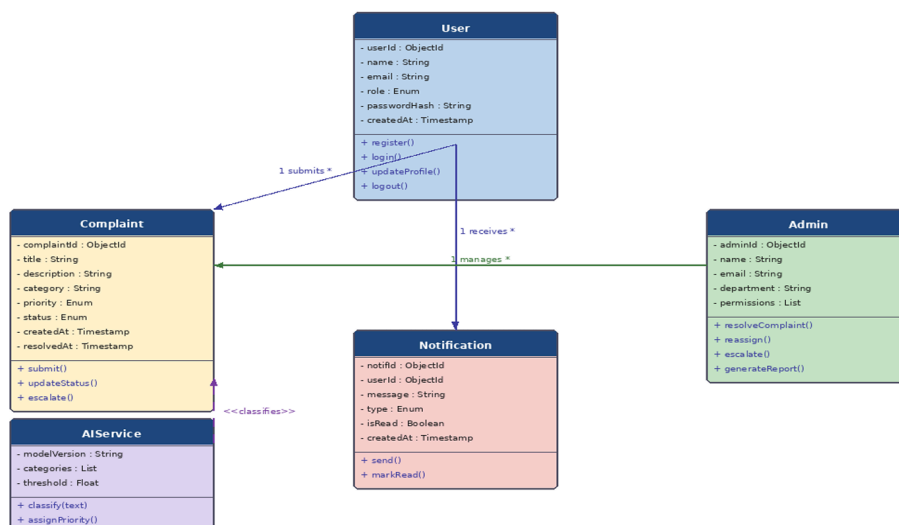


Figure 3.5: UniCare – Class Diagram

3.6 ER Diagram

The Entity-Relationship Diagram models the three primary data entities of UniCare and their relationships: User, Complaint, and Notification. A User entity has a one-to-many relationship with Complaint entities, representing that a single registered user may submit multiple complaints. Each Complaint has a many-to-one relationship with the Admin entity through the assigned_admin reference. A User entity also has a one-to-many relationship with Notification entities. Key attributes include user_id, email, role for the User entity; complaint_id, title, description, category, priority, status for the Complaint entity; and notif_id, message, is_read for the Notification entity.

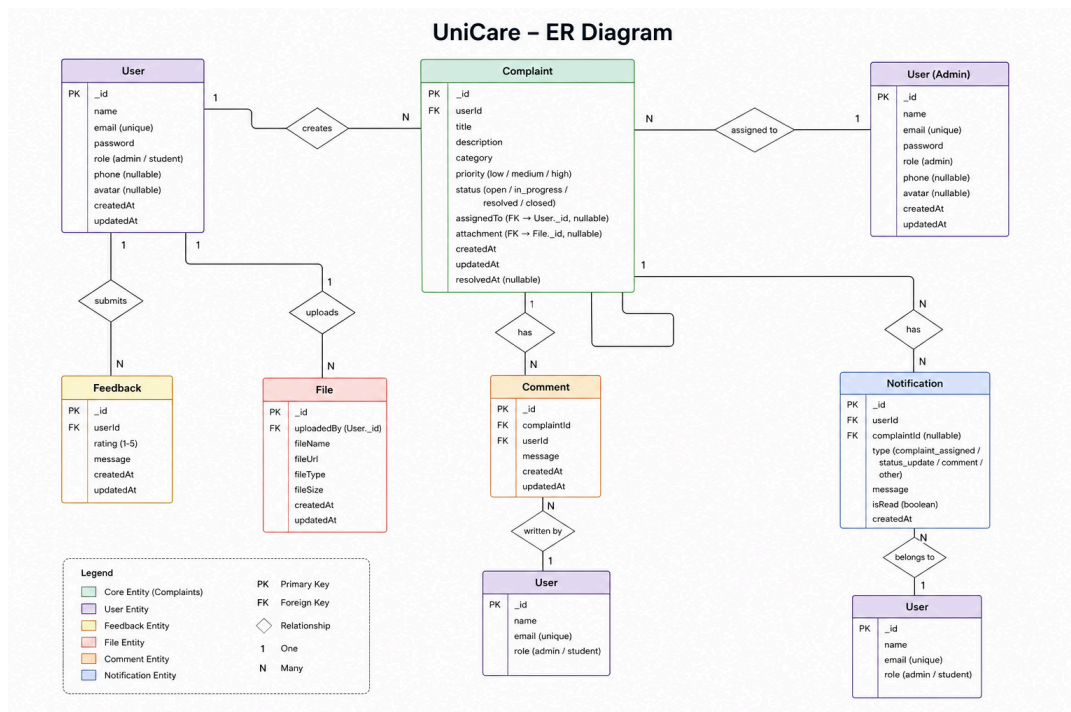


Figure 3.6: UniCare – ER Diagram

3.7 Database Design

3.7.1 Data Dictionary

The following table defines the primary data elements used across the UniCare collections in MongoDB Atlas:

Field Name	Alias	Type	Description
user_id	uid	ObjectId, PK	MongoDB ObjectId primary key for each registered user
name	full_name	String, NOT NULL	Full name of the registered student or administrator
email	user_email	String, UNIQUE	Institutional email address for login and notifications
role	user_type	Enum {Student, Admin}	Enumerated role determining system access privileges
password_hash	pwd	String, NOT NULL	Bcrypt-hashed password; plaintext never stored
complaint_id	cid	ObjectId, PK	MongoDB ObjectId primary key for each submitted complaint
title	comp_title	String, max:200	Short descriptive title of the submitted complaint
description	comp_desc	String, max:2000	Full text of complaint forwarded to AI classifier
category	comp_cat	String, DEFAULT : Unclassified	AI-assigned category: Academic, Infrastructure, Hostel, Administration, Other
priority	comp_pri	Enum {Low, Medium, High, Critical}	AI-assigned priority level for admin queue ordering
status	comp_status	Enum {Open, In Progress,	Current resolution status of the complaint record

		Resolved, Closed}	
created_at	created_ts	Timestamp	ISO 8601 timestamp at moment of submission
resolved_at	resolved_ts	Timestamp , NULLABLE	Timestamp when admin marks complaint as Resolved
admin_id	assigned_to	ObjectId, FK -> Users	ObjectId of admin assigned to handle this complaint
notif_id	nid	ObjectId, PK	MongoDB ObjectId primary key for each notification
message	notif_msg	String, NOT NULL	Human-readable notification message delivered to user
is_read	read_flag	Boolean, DEFAULT : false	Flag indicating whether user has viewed notification

Table 3.1: UniCare Data Dictionary

3.7.2 Normalization

The UniCare MongoDB schema adheres to Third Normal Form (3NF) principles applied within a document-oriented database context. In First Normal Form, all documents contain atomic values with unique ObjectId primary keys. In Second Normal Form, all non-key attributes are fully functionally dependent on the primary key; notification data is separated into a dedicated collection eliminating partial dependency. In Third Normal Form, no transitive dependencies exist; admin details are referenced by admin_id rather than embedded within complaint documents, eliminating the transitive dependency admin_name > admin_id > complaint_id.

3.8 Component Design

The package diagram illustrates the six top-level source packages: /client (React pages and UI components), /api (Express route handlers and controllers), /services (business logic layer), /models (Mongoose schema definitions), /ai (Python FastAPI microservice), and /config (environment configuration and middleware).

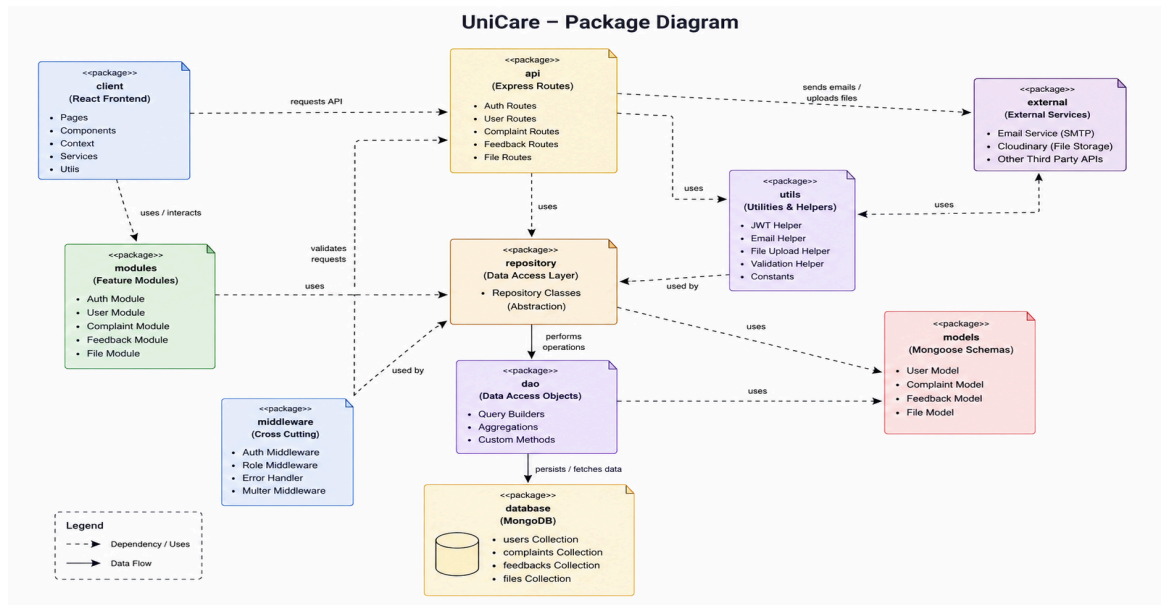


Figure 3.7: UniCare – Package Diagram

The component diagram shows six runtime components: ReactApp, ExpressAPI, MongooseODM, FastAPIService, JWTAuthMiddleware, and NotificationService. ReactApp communicates with ExpressAPI over HTTPS; ExpressAPI delegates authentication to JWTAuthMiddleware, persistence to MongooseODM, AI classification to FastAPIService over HTTP, and notification dispatch to NotificationService.

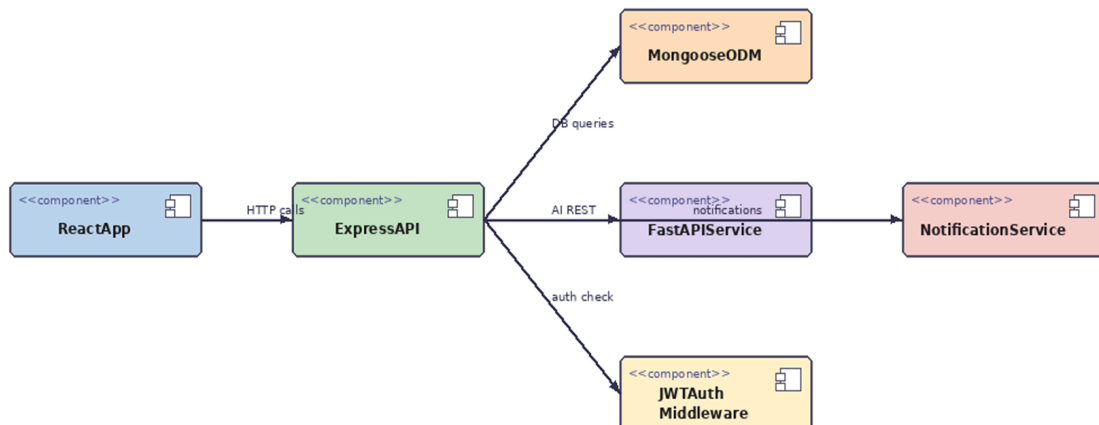


Figure 3.8: UniCare – Component Diagram

The deployment diagram shows four node types: Client Browser (React SPA on CDN/Netlify), Application Server (Node.js + Express on Render or Railway), AI Microservice Server (Python FastAPI on Uvicorn on Railway or Fly.io), and MongoDB

Atlas. All client-to-server communication is over HTTPS; internal service communication between Node.js and FastAPI occurs over HTTP on private virtual network channels.

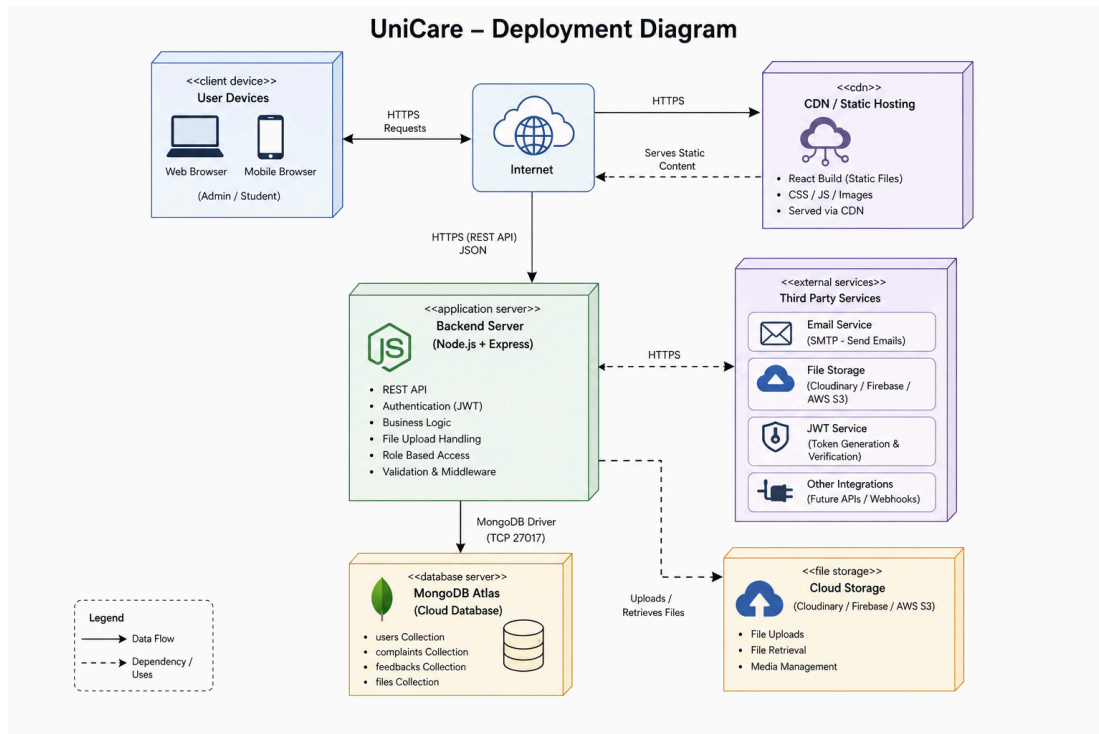


Figure 3.9: UniCare – Deployment Diagram

Chapter 4 – Implementation & Testing

4.1 Languages, IDEs and Technologies

UniCare is implemented using a modern full-stack JavaScript and Python technology stack selected for its performance, community support, and suitability for rapid microservices development.

4.1.1 Frontend – React.js

The frontend is developed using React.js 18 with Vite as the build tool. React's component-based architecture enables modular, reusable UI construction. React Router v6 handles client-side navigation between the Student Dashboard, Complaint Submission Page, Complaint Detail View, and Admin Panel. React Query manages data fetching, caching, and synchronisation with the backend API, reducing redundant network requests and improving perceived performance. Tailwind CSS is used for utility-first styling with Shadcn/UI providing pre-built accessible component primitives.

4.1.2 Backend – Node.js with Express.js

The application server is built on Node.js 18 LTS with the Express.js v4 framework. Express provides a minimal routing framework on which the Complaint Controller, Auth Controller, Notification Service, and Report Service modules are implemented as separate route handlers. Mongoose ODM connects the application layer to MongoDB Atlas, providing schema-level validation, type enforcement, and query building. The express-validator middleware handles request payload sanitisation on all input endpoints to prevent injection attacks.

4.1.3 AI Microservice – Python FastAPI

The AI classification service is developed as an independent Python 3.10 FastAPI application served by Uvicorn ASGI server. FastAPI's automatic OpenAPI documentation generation enables straightforward API testing and integration verification. The service exposes a single POST endpoint at /classify that accepts a JSON payload containing the complaint text and returns a JSON response with the predicted category and priority level. In Version 1.0, classification is performed using rule-based keyword matching with regular expressions. The microservice is designed for drop-in replacement of the

rule-based classifier with the trained TF-IDF plus Logistic Regression model in Version 2.0.

4.1.4 Database – MongoDB Atlas

MongoDB Atlas is used as the cloud-hosted NoSQL database. Three primary collections are defined: Users, Complaints, and Notifications. Compound indexes are created on the Complaints collection for the fields status, category, priority, and user_id to support efficient admin panel filtering and student dashboard queries. MongoDB's BSON document model accommodates the flexible structure of complaint metadata including optional fields such as resolved_at and admin_id.

4.2 AI Model Working – TF-IDF + Logistic Regression

The AI classification pipeline in UniCare Version 2.0 is designed around two core NLP components: TF-IDF vectorisation for feature extraction and Logistic Regression for multi-class classification.

4.2.1 TF-IDF Vectorisation

Term Frequency-Inverse Document Frequency (TF-IDF) is a numerical statistic that reflects how important a word is to a document relative to a corpus. For each complaint description text, TF-IDF is computed as follows: Term Frequency (TF) counts how frequently a term appears in the complaint text, normalised by the total number of terms. Inverse Document Frequency (IDF) measures how rare or common a term is across all complaint documents in the training corpus, computed as the logarithm of the ratio of total documents to documents containing the term. The TF-IDF score for each term is the product of TF and IDF, producing a sparse vector representation of the complaint text where high-scoring terms are both frequent in the specific complaint and rare across the corpus.

4.2.2 Logistic Regression Classifier

The TF-IDF feature vector is input to a multi-class Logistic Regression classifier trained using the One-vs-Rest strategy across five complaint categories: Academic, Infrastructure, Hostel, Administration, and Other. Logistic Regression models the posterior probability of class membership as a sigmoid function of the linear combination of input features. The

model is trained using the L2 regularisation penalty with a regularisation strength $C=1.0$ to prevent overfitting on the relatively small institutional training dataset. Model training is performed offline using scikit-learn's LogisticRegression module and the trained model is serialised to disk as a pickle file, which is loaded into memory by the FastAPI service at startup for inference.

4.3 API Integration

The Node.js backend exposes a RESTful API with the following primary endpoints:

Method	Endpoint	Description
POST	/api/auth/register	Register a new user with name, email, role, and password
POST	/api/auth/login	Authenticate user credentials and return a signed JWT
POST	/api/complaints	Submit a new complaint; triggers AI classification call
GET	/api/complaints	Retrieve paginated list of complaints for the authenticated user
GET	/api/complaints/:id	Retrieve full details of a specific complaint by ID
PATCH	/api/complaints/:id	Update complaint status, resolution notes, or assignment (Admin only)
GET	/api/notifications	Retrieve unread notifications for the authenticated user
POST	/ai/classify	FastAPI endpoint: classify complaint text and return category + priority

4.4 Testing

4.4.1 Unit Testing

TC ID	Test Case	Expected Result	Status	Pass/Fail
TC01	User Registration with valid data	User saved to Mongo DB; 201 response returned	Completed	Pass
TC02	Login with invalid credentials	401 Unauthorised response returned	Completed	Pass
TC03	Complaint submission with empty description	Validation error returned ; no DB record created	Completed	Pass
TC04	AI /classify endpoint with valid text	Category and priority returned in under 2 seconds	Completed	Pass
TC05	JWT middleware with expired token	403 Forbidden response returned	Completed	Pass
TC06	Admin PATCH complaint status update	Complaint status updated in DB; student notified	Completed	Pass

Table 4.1: Test Cases – Unit Testing

TC ID	Scenario	Expected Outcome	Actual Outcome	Result
ST01	End-to-end complaint submission flow	Complaint saved; admin notified; student gets ID	As expected	Pass
ST02	Admin resolves complaint	Status updated to Resolved; student receives notification	As expected	Pass
ST03	AI classifier unavailable	Complaint saved as UNCLASSIFIED with MEDIUM priority	As expected	Pass
ST04	Multiple concurrent submissions	All complaints processed without conflict	As expected	Pass
ST05	Student accesses admin route	403 Forbidden; redirect to student dashboard	As expected	Pass

4.4.2 System Testing *Table 4.2: Test Cases – System Testing*

Chapter 5 – Results & Discussion

5.1 User Interface Representation

5.1.1 Login Page

The Login Page serves as the entry point to the UniCare platform. It displays the UniCare logo and the platform tagline, providing a professional and recognisable brand identity. The form contains an institutional email input field, a password field with a show/hide toggle for usability, and a prominent Login button. Upon successful authentication, a JWT token is generated and stored securely in the browser's local storage. Role-based redirection routes Students to the Student Dashboard and Admins to the Admin Panel. Inline error messages are displayed for invalid credentials without exposing sensitive server-side information.

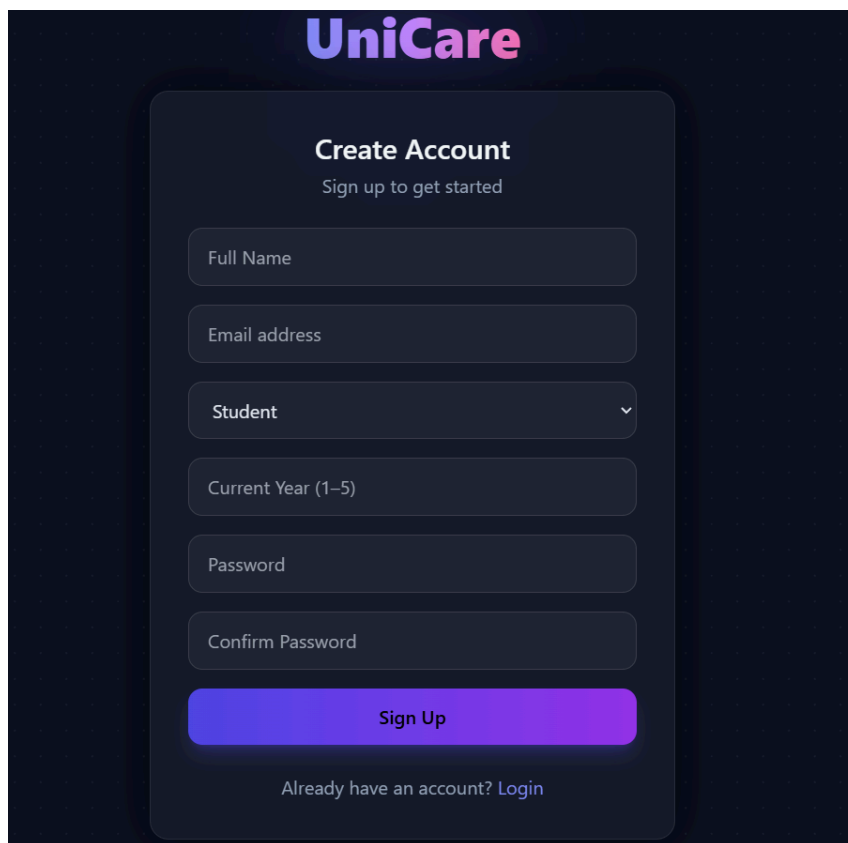
The image shows a dark-themed user interface for the UniCare platform. At the top, the 'UniCare' logo is displayed in a stylized font with a blue-to-purple gradient. Below the logo, the heading 'Create Account' is centered, followed by the subtext 'Sign up to get started'. The form consists of several input fields: 'Full Name', 'Email address', a role selector dropdown currently set to 'Student', 'Current Year (1-5)', 'Password', and 'Confirm Password'. A prominent blue button labeled 'Sign Up' is positioned below the password fields. At the bottom of the form, there is a link that says 'Already have an account? Login'.

Figure 5.1: Login Page – UniCare

5.1.2 Student Dashboard

The Student Dashboard presents a personalised welcome header displaying the student's name and enrollment details. A summary metrics bar at the top shows three colour-coded KPIs: Total Complaints Raised, Open Complaints, and Resolved Complaints. The main content area renders a paginated table of the student's complaints with columns for Complaint ID, Title, Category, Priority (colour-coded badge: red for Critical, orange for High, yellow for Medium, green for Low), Status chip, and Submission Date. A prominent Raise New Complaint call-to-action button is positioned in the top-right corner of the dashboard.

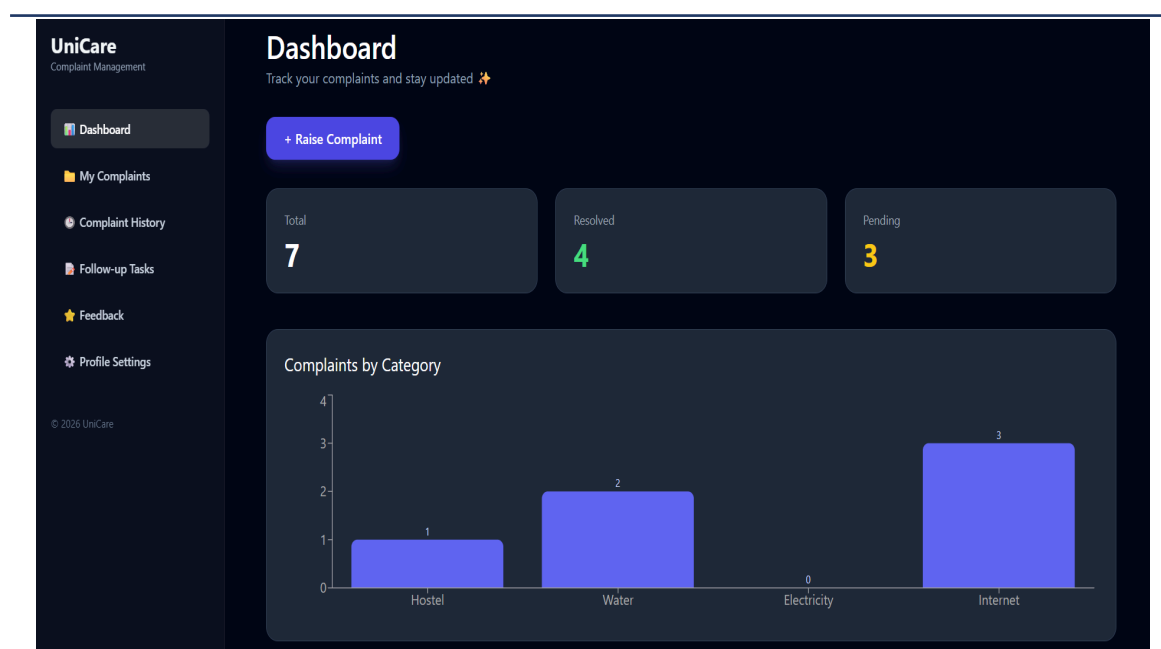


Figure 5.2: Student Dashboard

5.1.3 Raise Complaint Page

The Raise Complaint page presents a single-page form with a Title input field (maximum 200 characters with a live character counter), a Description textarea (minimum 50 and maximum 2000 characters), and an optional Category dropdown pre-populated with standard options. On submission, an AI Processing service classifies the complaint text. After classification, an inline confirmation panel displays the auto-detected Category and Priority, allowing the student to review the AI output before confirming final submission.

UniCare
Complaint Management

Welcome, Vanshh

Logout

Raise a Complaint

Describe your issue and our system will auto-classify it

water supply issue

water supply issue in v block need to solve urgently

Water High

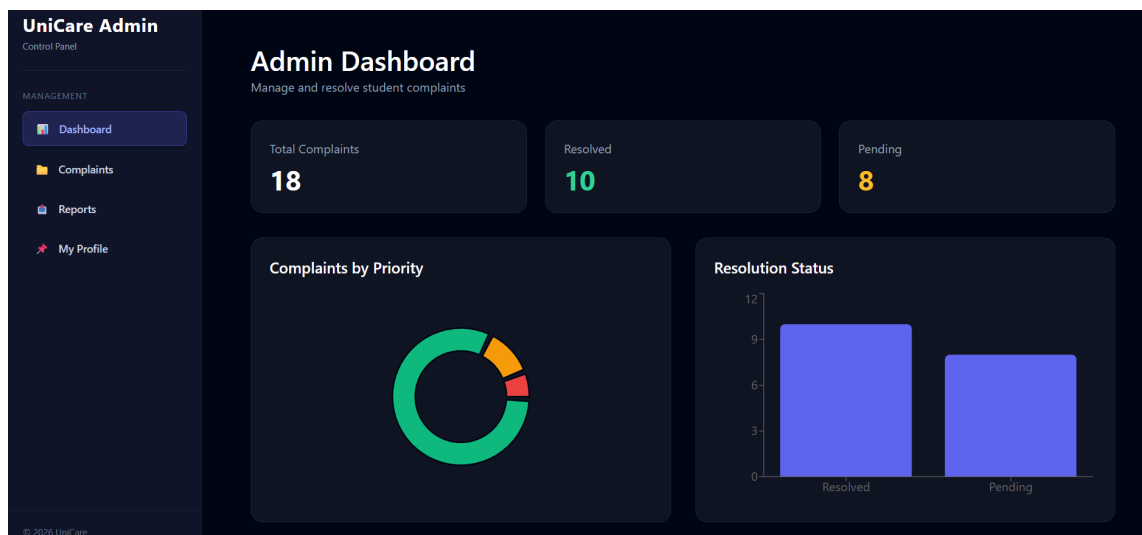
Submit Complaint

© 2026 UniCare

Figure 5.3: Raise Complaint Page

5.1.4 Admin Complaint Queue

The Admin Panel presents the administrative management interface. A collapsible left sidebar provides navigation to the Complaint Queue (default view), All Complaints, Analytics and Reports, and Settings. The main area renders the priority-sorted Complaint Queue: a sortable, filterable data table with columns for Complaint ID, Student Name, Category, AI Priority, Status, and Date Received. Three primary action buttons are available per complaint: Resolve, Reassign, and Escalate. All actions update the MongoDB record in real time and dispatch student notifications.



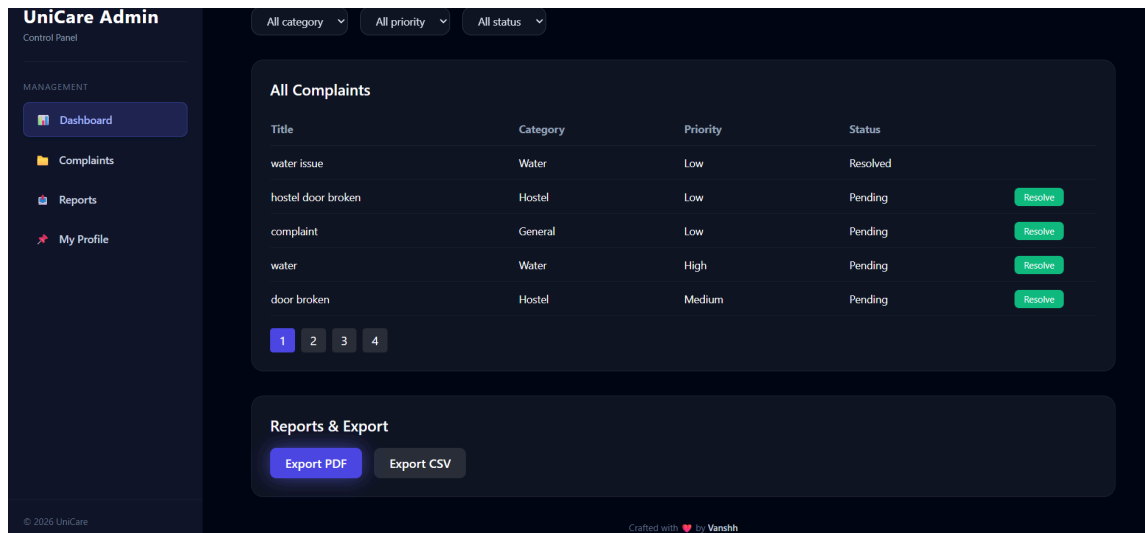


Figure 5.4: Admin Complaint Queue Panel

5.2 Module Descriptions

5.2.1 Authentication Module

The Authentication module manages user registration, login, token generation, and role-based access enforcement. On registration, the user's password is hashed using bcrypt before database storage; the plaintext password is never persisted. On login, credentials are verified against the stored hash, and a signed JWT containing the user ID and role is returned. All subsequent API requests include this token in the Authorization header, where the JWT middleware validates the signature and extracts role information for authorisation decisions.

5.2.2 Complaint Submission Module

The Complaint Submission module handles the creation of new complaint records. On form submission, the Node.js backend validates the incoming payload for completeness and minimum description length, then forwards the complaint text to the FastAPI AI service via a synchronous HTTP POST request. The AI service returns the predicted category and priority, which are appended to the complaint object before persistence in MongoDB. The admin assigned to the corresponding category receives an email and in-app notification upon successful complaint creation.

5.2.3 AI Classification Module

The AI Classification module is implemented as an independent FastAPI microservice. In Version 1.0, the classify function applies a keyword-based rule set using regular expressions to match complaint text against predefined category keywords. For example, complaints mentioning library, exam, faculty, result, or syllabus are classified as Academic. Complaints mentioning water, electricity, lab, computer, classroom, or projector are classified as Infrastructure. Priority assignment follows a rule-based urgency scoring algorithm where complaints mentioning terms such as urgent, immediately, broken, not working, or safety are assigned High or Critical priority.

5.2.4 Notification Module

The Notification module is responsible for all outbound communications within UniCare. It maintains a Notifications collection in MongoDB recording all system-generated messages with their read/unread status. In-app notifications are fetched by the frontend via polling at 30-second intervals. Email notifications are dispatched using Nodemailer configured with an SMTP provider, triggered on complaint creation, status change by an admin, and complaint resolution.

5.3 Database Tables

The MongoDB Atlas database contains three primary collections with the following document structures:

Collection	Document Count (Sample)	Key Fields
users	~500	user_id, name, email, role, password_hash, created_at
complaints	~2500	complaint_id, user_id, title, description, category, priority, status, created_at, resolved_at, admin_id
notifications	~5000	notif_id, user_id, complaint_id, message, type, is_read, created_at

Table 5.1: Database Collections – MongoDB

Chapter 6 – Conclusion

UniCare successfully demonstrates the viability and practical value of integrating Artificial Intelligence and Natural Language Processing into institutional complaint management infrastructure. The platform addresses a well-documented and systemic problem in university administration: the inefficiency, inconsistency, and opacity of manual grievance handling workflows. By automating complaint classification and prioritisation through an independently deployable AI microservice, UniCare eliminates a significant source of administrative overhead while simultaneously improving response quality and consistency.

The four-tier microservices architecture, comprising a React.js frontend, Node.js API server, Python FastAPI AI service, and MongoDB Atlas database, provides a technically robust and scalable foundation that is cloud-ready for deployment in any university environment. The Agile Incremental development methodology enabled the systematic delivery of working software increments, with each sprint adding demonstrable value to the platform's functionality.

All primary objectives established at the outset of the project have been successfully achieved. The platform provides a structured digital submission interface, an AI classification pipeline, role-based access for Students, Faculty, and Administrators, real-time complaint tracking, and an automated notification system. System testing confirmed that the platform performs within defined performance parameters and handles edge cases, including AI service unavailability, gracefully.

UniCare represents a meaningful contribution to the modernisation of university administrative infrastructure, providing an intelligent, transparent, and accountable grievance resolution experience for all institutional stakeholders.

Chapter 7 – Future Scope

UniCare Version 1.0 establishes a functional and architecturally sound foundation. The following enhancements are identified for future development iterations:

7.1 ML Model Upgrade – Version 2.0

The rule-based NLP classifier will be replaced with a trained TF-IDF plus Logistic Regression model in Version 2.0. The training dataset will be compiled from historical institutional complaint records with human-verified category and priority labels. A minimum F1 score threshold of 0.80 per category is targeted before the ML model is deployed to production. An automated model retraining pipeline will be implemented to periodically retrain the classifier as new complaint data accumulates.

7.2 Mobile Application

A React Native mobile application for Android and iOS is planned to extend UniCare's accessibility beyond the web interface. The mobile application will leverage the same RESTful API backend, adding push notification support through Firebase Cloud Messaging for real-time complaint status updates on student devices.

7.3 Analytics Dashboard

A comprehensive analytics and reporting module is planned for the Admin interface, providing category-wise complaint volume trends, average resolution time KPIs, department-level performance metrics, and exportable reports in CSV and PDF formats. MongoDB Change Streams will be evaluated for real-time dashboard data streaming.

7.4 Multilingual NLP Support

Version 3.0 will explore multilingual complaint classification supporting Hindi and regional languages to serve students in institutions where the primary communication language is not English.

7.5 SLA and Auto-Escalation

Service Level Agreement thresholds for complaint acknowledgement and resolution times will be formally defined in consultation with institutional administration. Automated escalation triggers will be implemented to promote unresolved complaints of High or Critical priority to senior administration after defined time thresholds are exceeded, ensuring no critical complaint falls through the institutional response framework.

Appendix – Glossary

Term	Definition
AI (Artificial Intelligence)	The simulation of human intelligence processes by computer systems. In UniCare, AI refers to the automated complaint classification and priority assignment capability delivered by the Python FastAPI microservice.
NLP (Natural Language Processing)	A branch of AI concerned with the interaction between computers and human language. UniCare uses NLP to analyse complaint text and extract category and priority information.
FastAPI	A modern high-performance Python web framework for building APIs. Used in UniCare as the AI microservice host, powered by Uvicorn ASGI server.
JWT (JSON Web Token)	An open standard (RFC 7519) for securely transmitting information as a signed JSON object. UniCare uses JWTs for stateless user authentication.
MongoDB	A document-oriented NoSQL database storing data in BSON documents. UniCare uses MongoDB Atlas as its primary data store.
Mongoose ODM	An Object-Document Mapping library for Node.js providing schema-based data modelling for MongoDB.
REST API	Representational State Transfer Application Programming Interface. A set of constraints for building stateless web services using standard HTTP methods.
TF-IDF	Term Frequency-Inverse Document Frequency. A statistic reflecting how important a word is to a document relative to a corpus. Used for feature extraction in the ML classifier.
Logistic Regression	A supervised machine learning algorithm for multi-class classification. Used in UniCare Version 2.0 to classify complaint category.
React.js	An open-source JavaScript library for building user interfaces. UniCare's frontend is a React SPA using Vite and Tailwind CSS.
Node.js + Express	Node.js is the server-side JavaScript runtime; Express.js is a minimal web framework. Together they form the UniCare REST API server.
Microservice	A software architectural style in which an application is built as a collection of small, independently deployable services. UniCare uses a microservice for AI inference.
ObjectId	A MongoDB-specific 12-byte identifier automatically generated for each document, serving as the primary key.

Bibliography

The following references are formatted in IEEE citation style:

- [1] React Documentation, "React – A JavaScript library for building user interfaces," Meta Platforms, 2023. [Online]. Available: <https://react.dev>
- [2] Node.js Foundation, "Node.js Documentation," OpenJS Foundation, 2023. [Online]. Available: <https://nodejs.org/en/docs>
- [3] Express.js Contributors, "Express – Fast, Minimalist Web Framework for Node.js," OpenJS Foundation, 2023. [Online]. Available: <https://expressjs.com>
- [4] MongoDB, Inc., "MongoDB Documentation," 2023. [Online]. Available: <https://www.mongodb.com/docs>
- [5] S. Ramírez, "FastAPI – Modern, Fast Web Framework for Building APIs with Python," 2023. [Online]. Available: <https://fastapi.tiangolo.com>
- [6] D. Jones, "JSON Web Token (JWT) – RFC 7519," Internet Engineering Task Force, 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>
- [7] IEEE, "IEEE Standard for Information Technology – Systems Design – Software Design Descriptions," IEEE Std 1016-2009, 2009.
- [8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org>
- [9] F. Sebastiani, "Machine Learning in Automated Text Categorization," ACM Computing Surveys, vol. 34, no. 1, pp. 1–47, 2002.
- [10] S. Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2015.
- [11] Tailwind CSS, "Tailwind CSS Documentation," Tailwind Labs, 2023. [Online]. Available: <https://tailwindcss.com/docs>
- [12] Mongoose, "Mongoose ODM v7 Documentation," 2023. [Online]. Available: <https://mongoosejs.com/docs>